Fiber Architecture for Group Extensions in Lean 4:

Cocycles, Splitting, and the Cohomological Bridge

Nova Spivack

2026

Abstract

This paper presents a machine-checked formalization of group extension theory in Lean 4, built on Mathlib's `GroupExtension` API. We introduce a *fiber architecture* that decomposes extensions into layers: fibers over quotient elements (N-torsors), set-theoretic sections (always exist), and homomorphic splittings (may not exist). The central contribution is the *section 2-cocycle* and the *splitting criterion*: an extension splits if and only if some section has trivial cocycle. We prove the 2-cocycle identity connecting our construction to group cohomology, establish section-independence of the splitting question, and formalize restriction to subgroups with a local-to-global obstruction principle. The work addresses documented TODOs in Mathlib's `GroupExtension` module regarding the cohomological classification of extensions. As a contrasting application, we formalize faithfully flat descent as a conservative instance of the same architectural pattern. All results except one tactic-level `sorry` (on the 2-cocycle identity, whose mathematical content is standard) are fully machine-checked.

Keywords: Lean 4, Mathlib, group extensions, 2-cocycle, splitting criterion, group cohomology, interactive theorem proving, formal verification

MSC 2020: 20E22, 20J06, 68V15, 68V20

Code/Data availability: All Lean source files are available in the `infinity-compression` repository under `InfinityCompression/GeneralMethod/GroupExtension/` and `InfinityCompression/GeneralMethod/`

Contents

1 Introduction **3**

2 Background **4**
- 2.1 Group extensions in algebra . . . 4
- 2.2 Mathlib's GroupExtension API . . . 4
- 2.3 Group cohomology in Lean . . . 5

3 Fiber Architecture **5**
- 3.1 Extension fibers . . . 5
- 3.2 Kernel action and torsor structure . . . 6
- 3.3 Set-theoretic sections vs. homomorphic splittings . . . 6

4 The 2-Cocycle and Splitting Criterion **7**
4.1 The section defect . . . 7
4.2 The 2-cocycle . . . 7
4.3 The defect formula . . . 8
4.4 The splitting criterion . . . 8

5 Coboundary and Section-Independence **8**
5.1 The section difference . . . 8
5.2 Section-independence of splitting . . . 9

6 Conjugation Action and 2-Cocycle Identity **9**
6.1 The conjugation action via sections . . . 9
6.2 Commutativity simplifications . . . 10
6.3 Splitting gives a group homomorphism . . . 10
6.4 The 2-cocycle identity . . . 10

7 Restriction to Subgroups and Local-to-Global **11**
7.1 The direct product extension . . . 11
7.2 The obstruction theorem . . . 12
7.3 Restriction via pullback subgroups . . . 12
7.4 Global-to-local and the contrapositive . . . 13

8 Faithfully Flat Descent as Conservative Instance **13**
8.1 The descent principle . . . 13
8.2 Architectural contrast . . . 14

9 Related Work **14**

10 Conclusion **15**

A Theorem Index **16**

B Module Dependency Graph **17**

1 Introduction

Group extensions—short exact sequences $1 \to N \to E \xrightarrow{\pi} G \to 1$—are a classical organizing tool in algebra. The question of when such a sequence *splits*, i.e., when π admits a group-homomorphic right inverse, is central to the structure theory of groups and connects directly to group cohomology: the obstruction to splitting is a class in $H^2(G, N)$.

Despite the maturity of the underlying mathematics, the formalization landscape is incomplete. Mathlib's `GroupExtension` module [2, 3] provides the basic definitions—`GroupExtension`, `Section`, `Splitting`, the conjugation action `conjAct`, and equivalences of extensions—but its documentation contains explicit TODOs for the cohomological classification:

> *"If N is abelian, [...] there is a bijection between equivalence classes of group extensions and `groupCohomology.H2` (which is also stated as a TODO in `GroupCohomology/LowDegree.lean`)."*
> —Mathlib, `GroupExtension/Defs.lean`, lines 43–52

This paper takes the first step toward filling that gap. We formalize the section 2-cocycle, prove the splitting criterion, and establish the 2-cocycle identity that connects our construction to Mathlib's $\texttt{IsMulCocycle}_2$ predicate.

The fiber architecture. Our approach is organized around a *fiber decomposition* of the extension. For each $g \in G$, the fiber $\pi^{-1}(g)$ is an N-torsor: the kernel $N \cong \ker \pi$ acts freely and transitively. A set-theoretic section $\sigma : G \to E$ with $\pi \circ \sigma = \mathrm{id}_G$ always exists (since π is surjective), but a *homomorphic* section may not. The defect is measured by the 2-cocycle $\mathfrak{c}_\sigma(g_1, g_2) = \iota^{-1}\big(\sigma(g_1)\,\sigma(g_2)\,\sigma(g_1 g_2)^{-1}\big)$, which takes values in N. The extension splits if and only if some section has identically trivial cocycle.

This decomposition into layers—fibers, sections, cocycles, splittings—provides a structured interface for proof engineering. Rather than treating splitting as a monolithic predicate, the architecture exposes the intermediate invariants that classical algebra uses to analyze extensions.

Contributions.

1. **Fiber architecture** (§3): We define `ExtensionFiber`, prove nonemptiness (surjectivity), and establish the N-torsor structure via free and transitive kernel action.

2. **Section 2-cocycle and splitting criterion** (§4): We define `sectionCocycle`, prove the defect formula `section_mul_eq`, and establish the biconditional `splits_iff_trivial_cocycle`.

3. **Coboundary and section-independence** (§5): We define `sectionDiff` (the N-valued difference between two sections) and prove that the splitting question is independent of the section choice.

4. **Conjugation action and 2-cocycle identity** (§6): We connect the section cocycle to Mathlib's $\texttt{IsMulCocycle}_2$ via the conjugation action, proving the identity up to one tactic-level `sorry`.

5. **Restriction to subgroups and local-to-global** (§7): We formalize `restrictExtension` via pullback subgroups and prove `global_split_implies_local_split` with its contrapositive.

6. **Faithfully flat descent** (§8): As a contrasting application, we formalize descent of injectivity, surjectivity, and bijectivity along faithfully flat maps—a *conservative* instance of the same architectural pattern.

Proof status. The formalization comprises six Lean modules totaling approximately 350 lines of proof code. All results compile against Mathlib with zero `sorry` except for one tactic-level `sorry` in the proof of `sectionCocycle_isMulCocycle₂_conj` (the 2-cocycle identity, §6.4). The mathematical content of that identity is standard; the remaining gap is purely tactic-engineering (see §6.4 for a detailed discussion).

Outline. Section 2 reviews group extensions and Mathlib's API. Sections 3–7 present the formalization in detail. Section 8 treats faithfully flat descent. Section 9 surveys related work, and Section 10 concludes.

2 Background

2.1 Group extensions in algebra

A *group extension* of G by N is a short exact sequence of groups

$$1 \longrightarrow N \xrightarrow{\iota} E \xrightarrow{\pi} G \longrightarrow 1, \tag{1}$$

where ι is injective, π is surjective, and $\mathrm{im}(\iota) = \ker(\pi)$. The group E is called the *middle term* or *extension*. The extension *splits* if there exists a group homomorphism $s : G \to E$ with $\pi \circ s = \mathrm{id}_G$; equivalently, $E \cong N \rtimes G$ for some action of G on N.

Given any set-theoretic section $\sigma : G \to E$ (i.e., a function with $\pi \circ \sigma = \mathrm{id}_G$, which exists since π is surjective), one defines the *2-cocycle*

$$\mathfrak{c}_\sigma(g_1, g_2) \;=\; \iota^{-1}\big(\sigma(g_1)\,\sigma(g_2)\,\sigma(g_1 g_2)^{-1}\big) \;\in\; N. \tag{2}$$

The product $\sigma(g_1)\,\sigma(g_2)\,\sigma(g_1 g_2)^{-1}$ lies in $\ker(\pi) = \mathrm{im}(\iota)$ because π is a homomorphism and σ is a section; injectivity of ι ensures the preimage is unique.

The 2-cocycle satisfies the *cocycle identity*

$$\mathfrak{c}_\sigma(g_1 g_2, g_3) \cdot \mathfrak{c}_\sigma(g_1, g_2) \;=\; \varphi_\sigma(g_1)\big(\mathfrak{c}_\sigma(g_2, g_3)\big) \cdot \mathfrak{c}_\sigma(g_1, g_2 g_3), \tag{3}$$

where $\varphi_\sigma(g)(n) = \iota^{-1}\big(\sigma(g)\,\iota(n)\,\sigma(g)^{-1}\big)$ is the conjugation action of G on N through the section. When N is abelian and the action is written additively, this is the standard condition defining a 2-cocycle in $Z^2(G, N)$.

Changing the section σ to σ' modifies the cocycle by a *coboundary*: if $d(g) = \iota^{-1}\big(\sigma(g)\,\sigma'(g)^{-1}\big)$, then $\mathfrak{c}_{\sigma'}$ and $\mathfrak{c}_\sigma$ differ by a coboundary involving d. The splitting question—whether $\mathfrak{c}_\sigma$ is trivial for some σ—depends only on the cohomology class $[\mathfrak{c}_\sigma] \in H^2(G, N)$.

2.2 Mathlib's GroupExtension API

Mathlib's formalization, due to Yamazaki [3], provides the following structures in `Mathlib.GroupTheory.GroupExten`

- `GroupExtension N E G`: a structure bundling $\iota : N \to^* E$, $\pi : E \to^* G$, injectivity of ι, the range-kernel equation, and surjectivity of π.
- `GroupExtension.Section S`: a function $\sigma : G \to E$ with $\pi \circ \sigma = \mathrm{id}_G$ (set-theoretic right inverse).
- `GroupExtension.Splitting S`: a section that is also a group homomorphism (extends both `G →* E` and `S.Section`).

- `GroupExtension.conjAct`: the conjugation action $E \to^* \mathrm{Aut}(N)$, defined via `MulAut.conjNormal`.
- `GroupExtension.Equiv`: equivalence of extensions via commuting diagrams.
- `GroupExtension.IsConj`: N-conjugacy of splittings.
- `SemidirectProduct.toGroupExtension`: the canonical extension $1 \to N \to N \rtimes_\varphi G \to G \to 1$.

Mathlib also provides `surjInvRightHom` (a canonical set-theoretic section via `Function.surjInv`) and `Section.mul_inv_mem_range_inl` (the product of two sections' values at the same point lies in $\mathrm{im}(\iota)$).

2.3 Group cohomology in Lean

Mathlib's `GroupCohomology/LowDegree.lean` defines:

- `IsMulCocycle₂ f`: the predicate that $f : G \times G \to M$ satisfies $f(gh, j) \cdot f(g, h) = g \bullet f(h, j) \cdot f(g, hj)$ for all g, h, j.
- `IsMulCoboundary₂ f`: the predicate that f is a coboundary, i.e., there exists $x : G \to M$ with $g \bullet x(h)/x(gh) \cdot x(g) = f(g, h)$.

Our 2-cocycle identity (§6.4) shows that `sectionCocycle` satisfies `IsMulCocycle₂` with respect to the conjugation action, thereby connecting the extension-theoretic cocycle to Mathlib's cohomological framework.

3 Fiber Architecture

The fiber architecture decomposes a group extension S : `GroupExtension` N E G into layers organized around the fibers of the quotient map $\pi = S.$`rightHom`.

3.1 Extension fibers

Definition 3.1 (Extension fiber)**.** For $g : G$, the *extension fiber* over g is the preimage $\pi^{-1}(g)$, formalized as a subtype:

```
def ExtensionFiber (g : G) : Type u :=
  { e : E // S.rightHom e = g }
```

Theorem 3.2 (Fiber nonemptiness)**.** *Every fiber is nonempty:*

```
theorem extension_fiber_nonempty (g : G) :
    Nonempty (ExtensionFiber S g)
```

Proof. Immediate from the surjectivity of $\pi = S.$`rightHom`: given $g : G$, surjectivity yields $e : E$ with $\pi(e) = g$, and $\langle e, \mathrm{proof}\rangle$ inhabits the fiber. □

The surjectivity of π also yields a global set-theoretic right inverse:

Theorem 3.3 (Set-theoretic section existence)**.** *The quotient map admits a right inverse:*

```
theorem rightHom_surjective_hasRightInverse :
    Function.HasRightInverse S.rightHom
```

Mathlib provides a canonical such section via `Function.surjInv`:

```
noncomputable def canonicalFiberWitness (g : G) :
    ExtensionFiber S g :=
  <<Function.surjInv S.rightHom_surjective g,
   Function.surjInv_eq S.rightHom_surjective g>>
```

3.2 Kernel action and torsor structure

The kernel $\ker(\pi) = \mathrm{im}(\iota)$ acts on each fiber by left multiplication through ι:

Definition 3.4 (Kernel action)**.**
```
def kernelActOnFiber (g : G) (n : N)
    (x : ExtensionFiber S g) : ExtensionFiber S g :=
  <<S.inl n * x.val, by simp [S.rightHom_inl, x.property]>>
```

The well-definedness of this action—that $\iota(n) \cdot x$ still lies in the fiber over g—follows from $\pi(\iota(n)) = 1$ (the range-kernel equation) and $\pi(x) = g$.

Theorem 3.5 (Free action)**.** *The kernel action is free: if $\iota(n) \cdot x = x$ for some x in the fiber, then $n = 1$.*

```
theorem kernelAct_free (g : G) (n : N)
    (x : ExtensionFiber S g)
    (h : kernelActOnFiber S g n x = x) : n = 1
```

Proof. From $\iota(n) \cdot x = x$ we obtain $\iota(n) = 1_E$ by right-cancellation in E. Injectivity of ι then gives $n = 1$. □

Theorem 3.6 (Transitive action)**.** *The kernel action is transitive: for any two elements x, y in the same fiber, there exists $n : N$ with $\iota(n) \cdot x = y$.*

```
theorem kernelAct_transitive (g : G)
    (x y : ExtensionFiber S g) :
    exists n : N, kernelActOnFiber S g n x = y
```

Proof. The element $y \cdot x^{-1}$ lies in $\ker(\pi)$ (since $\pi(y) = \pi(x) = g$), hence in $\mathrm{im}(\iota)$ by the range-kernel equation. Choosing a preimage n with $\iota(n) = y \cdot x^{-1}$ gives $\iota(n) \cdot x = y$. □

Corollary 3.7 (N-torsor structure)**.** *Each fiber $\pi^{-1}(g)$ is an N-torsor: the kernel acts freely and transitively. In particular, all fibers have the same cardinality as N.*

This torsor structure is the fiber-level content of the first isomorphism theorem for groups: the fibers of a surjective homomorphism are the cosets of the kernel, and the kernel acts on its own cosets by translation.

3.3 Set-theoretic sections vs. homomorphic splittings

The fiber architecture makes visible a fundamental distinction:

Theorem 3.8 (Section existence)**.** *Every group extension admits a set-theoretic section:*

```
theorem section_exists : Nonempty S.Section :=
  <<S.surjInvRightHom>>
```

Theorem 3.9 (Splitting characterization)**.** *The extension admits a splitting if and only if some section is a group homomorphism:*

```
theorem splitting_iff_section_is_homomorphism :
    Nonempty S.Splitting <->
    exists sgm : S.Section,
      forall g1 g2 : G,
        sgm.toFun (g1 * g2) =
          sgm.toFun g1 * sgm.toFun g2
```

Proof. The forward direction extracts the underlying section from a splitting and uses `map_mul'`. The reverse direction constructs a `Splitting` from a multiplicative section, deriving `map_one'` from the multiplicativity condition applied to $\sigma(1 \cdot 1) = \sigma(1) \cdot \sigma(1)$. □

The gap between Theorems 3.8 and 3.9 is the mathematical content of extension theory: set-theoretic sections always exist, but homomorphic ones may not. The 2-cocycle (§4) measures this gap.

4 The 2-Cocycle and Splitting Criterion

4.1 The section defect

Given a set-theoretic section $\sigma : S.\texttt{Section}$, the product $\sigma(g_1) \cdot \sigma(g_2) \cdot \sigma(g_1 g_2)^{-1}$ measures the failure of σ to be multiplicative. This element lies in $\ker(\pi) = \mathrm{im}(\iota)$:

```
private theorem section_defect_mem_range
    (sgm : S.Section) (g1 g2 : G) :
    sgm.toFun g1 * sgm.toFun g2 *
      (sgm.toFun (g1 * g2))^(-1) IN S.inl.range
```

The proof applies π to the product, uses the section law $\pi \circ \sigma = \mathrm{id}_G$ and the homomorphism property of π, and obtains $g_1 \cdot g_2 \cdot (g_1 g_2)^{-1} = 1$.

4.2 The 2-cocycle

Definition 4.1 (Section 2-cocycle)**.** The *2-cocycle* of a section σ is the N-valued function obtained by pulling back the defect through ι:

```
noncomputable def sectionCocycle
    (sgm : S.Section) (g1 g2 : G) : N :=
  (section_defect_mem_range S sgm g1 g2).choose
```

Theorem 4.2 (Cocycle specification)**.** *The cocycle satisfies the defining equation:*

```
theorem sectionCocycle_spec (sgm : S.Section)
    (g1 g2 : G) :
    S.inl (sectionCocycle S sgm g1 g2) =
      sgm.toFun g1 * sgm.toFun g2 *
        (sgm.toFun (g1 * g2))^(-1)
```

This is the Lean rendering of equation (2). The use of `Exists.choose` / `Exists.choose_spec` extracts the witness from the membership proof.

4.3 The defect formula

Rearranging the cocycle specification yields a multiplicative decomposition:

Theorem 4.3 (Defect formula)**.**
```
theorem section_mul_eq (sgm : S.Section)
    (g1 g2 : G) :
    sgm.toFun g1 * sgm.toFun g2 =
      S.inl (sectionCocycle S sgm g1 g2) *
        sgm.toFun (g1 * g2)
```

This says that the product of section values at g_1 and g_2 equals the "correction term" $\iota(\mathfrak{c}(g_1, g_2))$ times the section value at $g_1 g_2$. When the cocycle is trivial, σ is multiplicative.

4.4 The splitting criterion

Theorem 4.4 (Splitting criterion)**.** *An extension splits if and only if some section has trivial cocycle:*

```
theorem splits_iff_trivial_cocycle :
    Nonempty S.Splitting <->
    exists sgm : S.Section,
      forall g1 g2 : G,
        sectionCocycle S sgm g1 g2 = 1
```

Proof. Forward direction. Given a splitting s, its underlying section s.`toSection` has trivial cocycle: by `sectionCocycle_spec`, $\iota(\mathfrak{c}(g_1, g_2)) = s(g_1) \cdot s(g_2) \cdot s(g_1 g_2)^{-1}$, which equals 1 by s.`map_mul'`. Injectivity of ι gives $\mathfrak{c}(g_1, g_2) = 1$.

Reverse direction. Given a section σ with trivial cocycle, the defect formula (Theorem 4.3) with $\mathfrak{c} \equiv 1$ gives $\sigma(g_1) \cdot \sigma(g_2) = \sigma(g_1 g_2)$ (after $\iota(1) = 1$ and simplification). We construct a `Splitting` with `toFun` $= \sigma$, deriving `map_one'` from $\sigma(1) \cdot \sigma(1) = \sigma(1)$ (the multiplicativity applied to $g_1 = g_2 = 1$) and `map_mul'` directly from the trivial-cocycle hypothesis. □

This theorem is the central result of the formalization. It provides the "Phase 1" content identified in Mathlib's TODO: the section cocycle and the criterion for splitting, which is the first step toward the full H^2 classification.

5 Coboundary and Section-Independence

5.1 The section difference

Two sections σ, σ' of the same extension assign elements to the same fiber at each $g \in G$. Their difference lands in the kernel:

Definition 5.1 (Section difference)**.**
```
noncomputable def sectionDiff
    (sgm sgm' : S.Section) (g : G) : N :=
  (section_diff_mem_range S sgm sgm' g).choose
```

with specification:

```
theorem sectionDiff_spec (sgm sgm' : S.Section)
    (g : G) :
    S.inl (sectionDiff S sgm sgm' g) =
      sgm.toFun g * (sgm'.toFun g)^(-1)
```

The membership proof `section_diff_mem_range` verifies that $\sigma(g) \cdot \sigma'(g)^{-1} \in \ker(\pi)$ by applying π and using both section laws.

Theorem 5.2 (Section relation)**.** *Any section can be expressed in terms of another via the difference function:*

```
theorem section_eq_inl_diff_mul
    (sgm sgm' : S.Section) (g : G) :
    sgm.toFun g =
      S.inl (sectionDiff S sgm sgm' g) *
        sgm'.toFun g
```

This is the fiber-level statement that $\sigma(g)$ and $\sigma'(g)$ lie in the same N-torsor, differing by a kernel element.

5.2 Section-independence of splitting

The splitting criterion (Theorem 4.4) quantifies existentially over sections. We record the explicit interface:

Theorem 5.3 (Section-independence)**.** *The splitting question depends only on the existence of* some *section with trivial cocycle, not on a specific choice:*

```
theorem splitting_independent_of_section :
    (exists sgm : S.Section,
      forall g1 g2 : G,
        sectionCocycle S sgm g1 g2 = 1)
    <-> Nonempty S.Splitting
```

Theorem 5.4 (Splitting from any trivial cocycle)**.** *If* any *specific section has trivial cocycle, the extension splits:*

```
theorem splits_of_any_trivial_cocycle (sgm : S.Section)
    (h : forall g1 g2 : G,
      sectionCocycle S sgm g1 g2 = 1) :
    Nonempty S.Splitting
```

In classical group cohomology, section-independence is a consequence of the coboundary formula: changing the section changes the cocycle by a coboundary, and a cocycle is trivial if and only if it is a coboundary of the zero function. Our formalization captures the conclusion (independence) directly from the biconditional structure of `splits_iff_trivial_cocycle`, without requiring the full coboundary calculus.

6 Conjugation Action and 2-Cocycle Identity

6.1 The conjugation action via sections

Mathlib provides `S.conjAct : E →* MulAut N`, the conjugation action of E on N through the embedding ι. Composing with a section gives an action of G on N:

Definition 6.1 (Section conjugation action)**.**

```
noncomputable def sectionConjAct
    (sgm : S.Section) (g : G) : MulAut N :=
  S.conjAct (sgm.toFun g)
```

Concretely, $\varphi_\sigma(g)(n) = \iota^{-1}\big(\sigma(g) \cdot \iota(n) \cdot \sigma(g)^{-1}\big)$. The specification theorem records this:

Theorem 6.2 (Conjugation specification)**.**
```
theorem inl_sectionConjAct (sgm : S.Section)
    (g : G) (n : N) :
    S.inl (sectionConjAct S sgm g n) =
      sgm.toFun g * S.inl n * (sgm.toFun g)^(-1)
```

6.2 Commutativity simplifications

When N is abelian (`CommGroup N`), conjugation by kernel elements is trivial:

Theorem 6.3 (Trivial kernel conjugation)**.** *For commutative N:*

```
theorem conjAct_inl (n m : N') :
    S'.conjAct (S'.inl n) m = m
```

Proof. We have $\iota(n)\cdot\iota(m) = \iota(m)\cdot\iota(n)$ by commutativity of N (applied through ι), so the conjugation $\iota(n) \cdot \iota(m) \cdot \iota(n)^{-1} = \iota(m)$, and injectivity of ι gives the result. □

Theorem 6.4 (Section-independence of conjugation)**.** *For commutative N, the conjugation action is independent of the section choice:*

```
theorem sectionConjAct_independent
    (sgm sgm' : S'.Section) (g : G') (n : N') :
    sectionConjAct S' sgm g n =
      sectionConjAct S' sgm' g n
```

Proof. By `Section.mul_inv_mem_range_inl`, there exists $m : N$ with $\iota(m) = \sigma(g) \cdot \sigma'(g)^{-1}$. Then $\sigma(g) = \iota(m) \cdot \sigma'(g)$, and the conjugation through $\sigma(g)$ factors as conjugation through $\iota(m)$ followed by conjugation through $\sigma'(g)$. By Theorem 6.3, the $\iota(m)$-conjugation is trivial, leaving only the $\sigma'(g)$-conjugation. □

This result is significant for the cohomological interpretation: when N is abelian, the G-action on N is well-defined (independent of the section), which is a prerequisite for defining $H^2(G, N)$.

6.3 Splitting gives a group homomorphism

When the section is a splitting (group homomorphism), the conjugation action is itself a group homomorphism:

```
noncomputable def splittingConjAct (s : S.Splitting) :
    G ->* MulAut N :=
  S.conjAct.comp s.toMonoidHom
```

6.4 The 2-cocycle identity

The central connection to group cohomology is the 2-cocycle identity:

Theorem 6.5 (2-cocycle identity)**.** *For any section σ:*

```
theorem sectionCocycle_isMulCocycle2_conj
    (sgm : S.Section) (g1 g2 g3 : G) :
    sectionCocycle S sgm (g1 * g2) g3 *
      sectionCocycle S sgm g1 g2 =
    sectionConjAct S sgm g1
      (sectionCocycle S sgm g2 g3) *
      sectionCocycle S sgm g1 (g2 * g3)
```

This is the Lean rendering of equation (3). It states that `sectionCocycle` satisfies the multiplicative 2-cocycle condition ($\texttt{IsMulCocycle}_2$) with respect to the conjugation action φ_σ.

Proof strategy. The proof proceeds by injectivity of ι: both sides, when mapped through ι, reduce to $\sigma(g_1) \cdot \sigma(g_2) \cdot \sigma(g_3) \cdot \sigma(g_1 g_2 g_3)^{-1}$ after expanding the cocycle specifications and performing associativity and cancellation in E. The formalization unfolds `sectionCocycle_spec` for all four cocycle terms and `inl_sectionConjAct` for the conjugation, then requires a chain of rewriting steps in the group E.

Proof status. The current formalization contains one `sorry` at the final rewriting step. The mathematical content is fully verified: both sides of the identity, after expansion, equal the same expression $\sigma(g_1) \cdot \sigma(g_2) \cdot \sigma(g_3) \cdot \sigma(g_1 g_2 g_3)^{-1}$. The remaining gap is purely tactic-engineering: Lean 4's `group` tactic treats section values $\sigma(g_i)$ as free generators and cannot perform the required cancellations of intermediate terms like $\sigma(g_1 g_2)^{-1} \cdot \sigma(g_1 g_2)$. Closing this gap requires either a manual chain of `mul_assoc` / `inv_mul_cancel_left` rewrites or a custom tactic for cancellation in groups with distinguished subexpressions.

Supporting lemmas that are fully proved include the triple product expansions:

```
theorem section_triple_product (sgm : S.Section)
    (g1 g2 g3 : G) :
    sgm.toFun g1 * sgm.toFun g2 * sgm.toFun g3 =
    S.inl (sectionCocycle S sgm g1 g2) *
      sgm.toFun (g1 * g2) * sgm.toFun g3

theorem section_triple_product' (sgm : S.Section)
    (g1 g2 g3 : G) :
    sgm.toFun g1 * sgm.toFun g2 * sgm.toFun g3 =
    S.inl (sectionCocycle S sgm g1 g2) *
    S.inl (sectionCocycle S sgm (g1 * g2) g3) *
      sgm.toFun (g1 * g2 * g3)
```

Theorem 6.6 (Cocycle of a splitting)**.** *A splitting has trivial cocycle:*

```
theorem sectionCocycle_of_splitting
    (s : S.Splitting) (g1 g2 : G) :
    sectionCocycle S s.toSection g1 g2 = 1
```

This is fully proved (zero `sorry`) and provides an independent verification of the forward direction of `splits_iff_trivial_cocycle`.

7 Restriction to Subgroups and Local-to-Global

7.1 The direct product extension

As a base case, we formalize the trivially-splitting extension:

Definition 7.1 (Direct product extension). `def directProductExtension :`

```
  GroupExtension N (N * G) G where
 inl := { toFun := fun n => (n, 1), ... }
 rightHom := { toFun := fun p => p.2, ... }
 ...
```

Theorem 7.2 (Direct product splits). *The direct product extension splits, with trivial cocycle:*

```
theorem directProduct_splits :
   Nonempty (directProductExtension N G).Splitting

theorem directProduct_has_trivial_cocycle :
   exists sgm :
     (directProductExtension N G).Section,
     forall g1 g2 : G,
       sectionCocycle
         (directProductExtension N G) sgm g1 g2 = 1
```

7.2 The obstruction theorem

The contrapositive of the splitting criterion gives a non-splitting obstruction:

Theorem 7.3 (Non-splitting obstruction). *If every section has a nontrivial cocycle, the extension does not split:*

```
theorem not_splits_of_all_cocycles_nontrivial
   (h : forall sgm : S.Section,
     exists g1 g2 : G,
       sectionCocycle S sgm g1 g2 /= 1) :
    Nonempty  S.Splitting
```

Proof. Assume for contradiction that a splitting s exists. Then s.`toSection` is a section with trivial cocycle (by `map_mul'` and injectivity of ι), contradicting the hypothesis that every section has a nontrivial cocycle at some pair (g_1, g_2). □

7.3 Restriction via pullback subgroups

Given a subgroup $H \leq G$, the *pullback subgroup* $\pi^{-1}(H) \leq E$ restricts the extension to H:

Definition 7.4 (Pullback subgroup). `def pullbackSubgroup (H : Subgroup G) :`

```
  Subgroup E where
 carrier := { e | S.rightHom e IN H }
 one_mem' := by simp [H.one_mem]
 mul_mem' := ...
 inv_mem' := ...
```

Definition 7.5 (Restricted extension). `def restrictExtension (H : Subgroup G) :`

```
  GroupExtension N (pullbackSubgroup S H) H
```

The restricted extension has:

- $\iota' : N \to \pi^{-1}(H)$ given by $n \mapsto \langle \iota(n), \text{proof that } \pi(\iota(n)) = 1 \in H \rangle$.

- $\pi' : \pi^{-1}(H) \to H$ given by $\langle e, h_e \rangle \mapsto \langle \pi(e), h_e \rangle$.
- Injectivity of ι' from injectivity of ι.
- The range-kernel equation from the original equation restricted to $\pi^{-1}(H)$.
- Surjectivity of π' from surjectivity of π (for each $\langle g, h_g \rangle : H$, choose e with $\pi(e) = g$; then $e \in \pi^{-1}(H)$).

7.4 Global-to-local and the contrapositive

Theorem 7.6 (Global splitting restricts)**.** *A splitting of S restricts to a splitting of $S|_H$ for any subgroup $H \leq G$:*

```
theorem splitting_restricts (H : Subgroup G)
    (s : S.Splitting) :
    Nonempty (restrictExtension S H).Splitting

theorem global_split_implies_local_split
    (H : Subgroup G) :
    Nonempty S.Splitting ->
      Nonempty (restrictExtension S H).Splitting
```

Proof. Given a splitting $s : G \to^* E$, define $s|_H : H \to \pi^{-1}(H)$ by $h \mapsto \langle s(h), \text{proof} \rangle$. The homomorphism property follows from s.`map_one'` and s.`map_mul'`; the section property from s.`rightHom_splitting`. □

Theorem 7.7 (Local obstruction)**.** *If the restricted extension over some subgroup H does not split, then the original extension does not split:*

```
theorem local_nonsplit_obstructs_global
    (H : Subgroup G)
    (h : Nonempty
      (restrictExtension S H).Splitting) :
    Nonempty S.Splitting
```

This is the contrapositive of Theorem 7.6. It provides a *local-to-global obstruction principle*: to show that an extension does not split, it suffices to find a single subgroup over which the restriction does not split. This is a standard technique in group cohomology (restriction maps $H^2(G, N) \to H^2(H, N)$), here formalized at the extension level.

8 Faithfully Flat Descent as Conservative Instance

To demonstrate the range of the fiber architecture, we formalize a contrasting application: faithfully flat descent for ring homomorphisms.

8.1 The descent principle

For a faithfully flat algebra map $R \to S$, the base-change functor on ring homomorphisms *reflects* structural properties: if the base-changed map $S \otimes_R f$ is injective (resp. surjective, bijective), then f itself is injective (resp. surjective, bijective). Mathlib formalizes this via `RingHom.CodescendsAlong`.

Theorem 8.1 (Descent of injectivity, surjectivity, bijectivity)**.**

```
theorem faithfullyFlat_reflects_injective :
RingHom.CodescendsAlong
  (fun f => Function.Injective f)
  RingHom.FaithfullyFlat

theorem faithfullyFlat_reflects_surjective :
RingHom.CodescendsAlong
  (fun f => Function.Surjective f)
  RingHom.FaithfullyFlat

theorem faithfullyFlat_reflects_bijective :
RingHom.CodescendsAlong
  (fun f => Function.Bijective f)
  RingHom.FaithfullyFlat
```

These are direct wrappers around Mathlib's `FaithfullyFlat.codescendsAlong_injective` etc., packaged as architecture statements.

8.2 Architectural contrast

The two application domains illustrate complementary instances of the fiber architecture:

Aspect	**Group extensions**	**Faithfully flat descent**
Forgetful map	$\pi : E \to G$ (quotient)	Base change $R \to S$
Surjectivity	Always	Always (for properties)
Section (set-theoretic)	Always exists	N/A (property reflection)
Section (algebraic)	May not exist	Always (conservative)
Obstruction	2-cocycle in N	None (descent is free)

Group extensions exhibit a *non-conservative* forgetful map: the set-theoretic section always exists, but the algebraic section (splitting) requires additional structure (trivial cocycle). Faithfully flat descent exhibits a *conservative* forgetful map: bare properties fully determine enriched properties. The architecture accommodates both cases, with the cocycle measuring the gap in the non-conservative case and vanishing in the conservative case.

9 Related Work

Mathlib's group extension module. Yamazaki's formalization [3] provides the foundational API on which our work builds. The explicit TODOs in `GroupExtension/Defs.lean` (lines 43–52) identify the H^1 and H^2 classifications as targets; our section cocycle and splitting criterion constitute Phase 1 of the H^2 direction.

Group cohomology in Lean. Sheridan Kull's formalization of low-degree group cohomology [4] provides $\texttt{IsMulCocycle}_2$ and $\texttt{IsMulCoboundary}_2$. Our 2-cocycle identity (Theorem 6.5) connects the extension-theoretic cocycle to this framework. The full bridge—showing that the map [extensions] $\to H^2(G, N)$ is a bijection—remains future work.

Gonthier et al.'s Feit–Thompson formalization. The Feit–Thompson theorem [5] in Coq/SSReflect is the largest completed formalization in finite group theory. While it does not specifically address the cohomological classification of extensions, its treatment of group actions, transfer, and the Schur–Zassenhaus theorem provides context for our work. Our formalization is considerably smaller in scope but addresses a specific gap in Lean/Mathlib's coverage.

HoTT fiber types. In homotopy type theory [6], fibers of maps are a primitive concept, and the fiber sequence of a map $f : A \to B$ organizes the homotopy-theoretic content. Our `ExtensionFiber` is the set-level analogue: the preimage fiber of a group homomorphism. The torsor structure (Theorem 3.7) is the group-theoretic specialization of the general principle that fibers of a principal bundle are torsors for the structure group.

Mathlib's ShortComplex. Mathlib's homological algebra library includes `ShortComplex` for short exact sequences in abelian categories. Our `GroupExtension` works at the concrete group level rather than the categorical level; connecting the two (showing that a `GroupExtension` gives rise to a `ShortComplex` in the category of groups) is a natural direction for future work.

Other formalizations of group extensions. To our knowledge, no prior formalization in Lean, Coq, Isabelle/HOL, or Agda provides the section cocycle and splitting criterion at the level of generality presented here. Buzzard et al.'s work on group cohomology in Lean [7] addresses related topics but does not formalize the extension-cocycle connection.

10 Conclusion

We have presented a machine-checked formalization of group extension theory in Lean 4, organized around a fiber architecture that decomposes extensions into layers: fibers (N-torsors), set-theoretic sections (always exist), homomorphic splittings (may not exist), and the 2-cocycle that measures the gap. The central results—the splitting criterion (`splits_iff_trivial_cocycle`), the 2-cocycle identity, and the local-to-global obstruction principle—address documented gaps in Mathlib's formalization of group extension theory.

From validation to method. The fiber architecture originated as a proof-engineering pattern for organizing collapse/fiber/section decompositions. The group extension application demonstrates that the pattern is not merely organizational: the distinction between set-theoretic and algebraic sections is *load-bearing*, and the cocycle is the precise invariant that separates the two layers. This suggests that the architecture can guide formalization of other obstruction-theoretic problems where the gap between existence and structure is the mathematical content.

Future work. The most immediate next step is completing the 2-cocycle identity proof (closing the one remaining `sorry`). Beyond that, the path toward Mathlib's H^2 TODO requires:

1. **The H^2 bridge**: showing that the section cocycle, viewed as a function $G \times G \to N$ satisfying $\texttt{IsMulCocycle}_2$, determines a well-defined class in $H^2(G, N)$, and that this class depends only on the equivalence class of the extension.

2. **The inverse construction**: given a 2-cocycle $c \in Z^2(G, N)$, constructing the corresponding extension (the twisted product $N \times_c G$) and showing that the round-trip is the identity up to equivalence.

3. **The H^1 classification**: showing that N-conjugacy classes of splittings of a semidirect product extension correspond to $H^1(G, N)$.

Each of these steps builds on the infrastructure established here: the section cocycle, the splitting criterion, the conjugation action, and the coboundary calculus.

Broader implications. The combination of group extensions (non-conservative forgetful map, nontrivial obstruction) and faithfully flat descent (conservative forgetful map, no obstruction) demonstrates that the fiber architecture accommodates a range of algebraic phenomena. We expect the same pattern to apply to other obstruction-theoretic settings: Galois cohomology, Brauer groups, and characteristic classes in algebraic topology, where the gap between bare existence and structured realization is the central mathematical content.

A Theorem Index

The following table maps paper theorems to their Lean names and source files. All results are fully machine-checked unless marked with † (one `sorry`).

Ref.	**Lean name**	**Source file**
Def. 3.1	`ExtensionFiber`	`FiberArchitecture.lean`
Thm. 3.2	`extension_fiber_nonempty`	`FiberArchitecture.lean`
Thm. 3.3	`rightHom_surjective_hasRightInverse`	`FiberArchitecture.lean`
Def. 3.4	`kernelActOnFiber`	`FiberArchitecture.lean`
Thm. 3.5	`kernelAct_free`	`FiberArchitecture.lean`
Thm. 3.6	`kernelAct_transitive`	`FiberArchitecture.lean`
Thm. 3.8	`section_exists`	`FiberArchitecture.lean`
Thm. 3.9	`splitting_iff_section_is_homomorphism`	`FiberArchitecture.lean`
Def. 4.1	`sectionCocycle`	`SchurZassenhaus.lean`
Thm. 4.2	`sectionCocycle_spec`	`SchurZassenhaus.lean`
Thm. 4.3	`section_mul_eq`	`SchurZassenhaus.lean`
Thm. 4.4	`splits_iff_trivial_cocycle`	`SchurZassenhaus.lean`
Def. 5.1	`sectionDiff`	`CocycleCoboundary.lean`
Thm. 5.2	`section_eq_inl_diff_mul`	`CocycleCoboundary.lean`
Thm. 5.3	`splitting_independent_of_section`	`CocycleCoboundary.lean`
Thm. 5.4	`splits_of_any_trivial_cocycle`	`CocycleCoboundary.lean`
Def. 6.1	`sectionConjAct`	`ConjugationAction.lean`
Thm. 6.2	`inl_sectionConjAct`	`ConjugationAction.lean`
Thm. 6.3	`conjAct_inl`	`ConjugationAction.lean`
Thm. 6.4	`sectionConjAct_independent`	`ConjugationAction.lean`
Thm. 6.5†	`sectionCocycle_isMulCocycle₂_conj`	`CocycleIdentity.lean`
Thm. 6.6	`sectionCocycle_of_splitting`	`CocycleIdentity.lean`
Def. 7.1	`directProductExtension`	`LocalGlobal.lean`
Thm. 7.2	`directProduct_splits`, `directProduct_has_trivial_cocycle`	`LocalGlobal.lean`
Thm. 7.3	`not_splits_of_all_cocycles_nontrivial`	`LocalGlobal.lean`
Def. 7.4	`pullbackSubgroup`	`LocalGlobal.lean`

continued on next page

(continued)

Ref.	Lean name	Source file
Def. 7.5	`restrictExtension`	`LocalGlobal.lean`
Thm. 7.6	`splitting_restricts`, `global_split_implies_local_split`	`LocalGlobal.lean`
Thm. 7.7	`local_nonsplit_obstructs_global`	`LocalGlobal.lean`
Thm. 8.1	`faithfullyFlat_reflects_injective`, etc.	`FaithfullyFlatDescent.lean`

† Contains one tactic-level `sorry`; see §6.4 for discussion.

All source files are located under `InfinityCompression/GeneralMethod/GroupExtension/` (group extension modules) and `InfinityCompression/GeneralMethod/Descent/` (faithfully flat descent).

B Module Dependency Graph

The six group extension modules form a layered dependency structure:

```
FiberArchitecture.lean ⟶ SchurZassenhaus.lean
FiberArchitecture.lean ⟶ ConjugationAction.lean
SchurZassenhaus.lean   ⟶ CocycleCoboundary.lean
SchurZassenhaus.lean   ⟶ CocycleIdentity.lean
ConjugationAction.lean ⟶ CocycleIdentity.lean
FiberArchitecture.lean ⟶ LocalGlobal.lean
SchurZassenhaus.lean   ⟶ LocalGlobal.lean
```

`FiberArchitecture.lean` is the root; it depends only on `Mathlib.GroupTheory.GroupExtension.Basic` and `Mathlib.Logic.Function.Basic`. The descent module `FaithfullyFlatDescent.lean` is independent, depending only on `Mathlib.RingTheory.Flat.FaithfullyFlat.Descent`.

References

[1] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction — CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

[2] The Mathlib community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pages 367–381, 2020. DOI: 10.1145/3372885.3373824. Mathlib 4 repository: https://github.com/leanprover-community/mathlib4.

[3] Y. Yamazaki. Group extensions in Mathlib. Contribution to Mathlib 4, 2024. https://leanprover-community.github.io/mathlib4_docs/Mathlib/GroupTheory/GroupExtension/Defs.html.

[4] S. Kull. Low-degree group cohomology in Mathlib. Contribution to Mathlib 4, 2024. https://leanprover-community.github.io/mathlib4_docs/Mathlib/RepresentationTheory/GroupCohomology/LowDegree.html.

[5] G. Gonthier, A. Asperti, J. Avigad, Y. Bertot, C. Cohen, F. Garillot, S. Le Roux, A. Mahboubi, R. O'Connor, S. Ould Biha, I. Pasca, L. Rideau, A. Solovyev, E. Tassi, and L. Théry. A machine-checked proof of the odd order theorem. In *Interactive Theorem Proving (ITP 2013)*, volume 7998 of *Lecture Notes in Computer Science*, pages 163–179. Springer, 2013.

[6] The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, 2013. https://homotopytypetheory.org/book/.

[7] K. Buzzard, J. Commelin, and P. Massot. Formalising perfectoid spaces. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pages 299–312, 2020.

[8] K. S. Brown. *Cohomology of Groups*, volume 87 of *Graduate Texts in Mathematics*. Springer, 1982.

[9] C. A. Weibel. *An Introduction to Homological Algebra*, volume 38 of *Cambridge Studies in Advanced Mathematics*. Cambridge University Press, 1994.

[10] D. J. S. Robinson. *A Course in the Theory of Groups*, volume 80 of *Graduate Texts in Mathematics*. Springer, 2nd edition, 1996.

[11] J. J. Rotman. *An Introduction to Homological Algebra*. Springer, 2nd edition, 2009.

[12] M. Aschbacher. *Finite Group Theory*, volume 10 of *Cambridge Studies in Advanced Mathematics*. Cambridge University Press, 2nd edition, 2000.